Mohit Shah, Maanas Goel
Professor Mangharam
ESE 3600 Final Project Report
December 12, 2024

PlateMate: Automated Nutrient Tracking System

Introduction

PlateMate is an automated nutrient tracking system designed to analyze food plates and provide detailed nutritional information, making it easier for users to monitor their meals. By simply uploading a picture of a meal, users can instantly receive insights into key nutrients such as calories, protein, carbohydrates, and sugars. Built using a carefully curated dataset, an efficient machine learning model, and a user-friendly interface, PlateMate ensures both accuracy and accessibility for its users.

The application is designed with practicality in mind, making it suitable for use in schools, homes, or by individuals seeking to monitor their nutritional intake. While the current model is trained and tested on six food categories, the system has the potential to expand to a wider range of food items, offering comprehensive feedback for diverse meal types. PlateMate promotes healthier eating habits in an easy-to-use format, highlighting how machine learning can be used to improve health outcomes and effectively manage nutrition.

Motivation

Diet plays a critical role in maintaining overall health. Yet many individuals, particularly children, lack access to the resources needed to monitor and manage their nutritional intake. In schools, children are often served meals that are not nutritionally balanced. Children often prefer junk food over healthier options, leading to poor dietary habits and contributing to long-term health problems such as obesity, diabetes, and malnutrition. These challenges highlight the importance of providing tools that can assist in making informed dietary choices.

PlateMate was developed to address this gap by offering a user-friendly platform to analyze food plates and track nutritional content of meals. It is particularly useful for parents and educators to monitor the nutritional intake of children, ensuring they consume balanced and healthy meals. PlateMate aims to not only promote better eating habits, but also support long-term wellness, especially for the future generations.

Data Collection

The dataset for PlateMate was carefully created to ensure it was diverse and well-representative for training our machine learning model. We collected a total of 480 images and divided them into six food categories: Apples, Oranges, Bananas, Chips, Cookies, and Yogurt. For each category, we included 80 images with variations such as Apples including both red and green apples, Oranges and Bananas of varying sizes, Chips representing six different flavors (Red Doritos, Blue Doritos, Yellow Lays, Fritos, Cheetos, Ruffles), Cookies including three distinct types (Oreos, Chips Ahoy, Nutter Butter), and Yogurt featuring three Chobani flavors (Mango, Strawberry, Key Lime).

To ensure the model could handle real-world scenarios, we captured images from multiple angles and orientations, such as front-facing, side views, top-down perspectives, and tilted angles. This was important so that our model could accurately recognize food items even when they appeared sideways, upside-down, or in irregular positions. For instance, a bag of chips placed on a plate could be in any orientation, and we wanted our model to confidently detect it regardless of its position. By including these variations, we made our dataset more robust and diverse.

In addition to the original dataset, we applied several data augmentation techniques to further expand our training dataset. These techniques included rescaling pixel values to normalize the images, rotating them to simulate different alignments, shifting them horizontally and vertically, zooming in and out to account for different scales, and flipping them horizontally to introduce orientation changes. These augmentations effectively increased the diversity of the training data without requiring additional image collection, enabling the model to generalize better across unseen scenarios.

Finally, the dataset was split into three subsets; 70% of the images (336 images) for training, 15% (72 images) for validation during training to monitor its performance, and the remaining 15% (72 images) for testing to evaluate the model's accuracy on entirely unseen data.



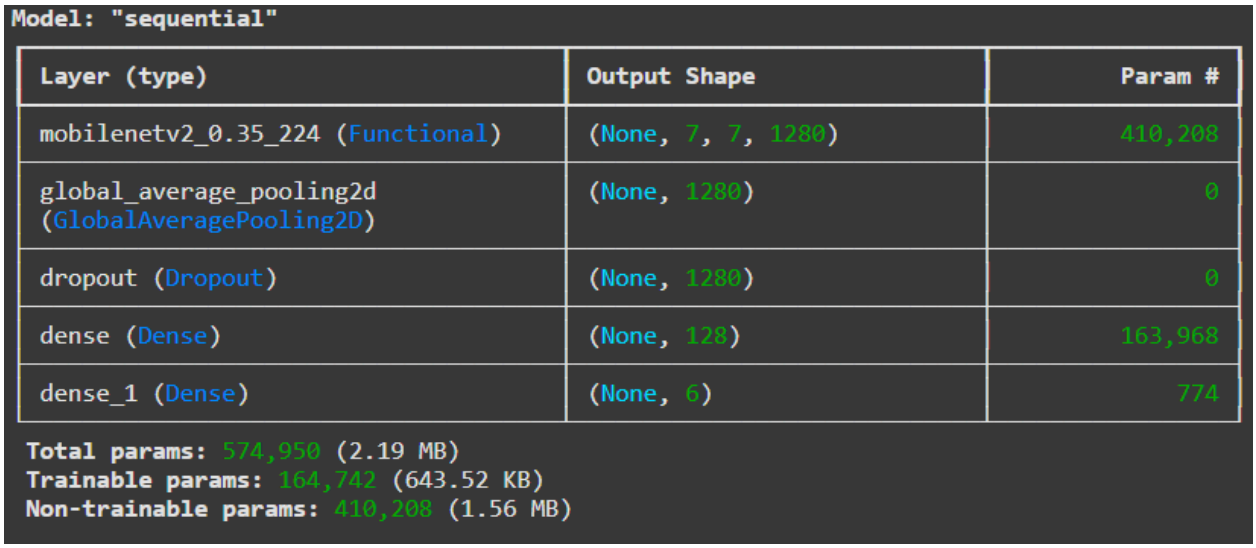
Figure 1: Sample Images from the Dataset

Code Generation and Model Choice

First, we wrote a Python script on Colab [https://colab.research.google.com/drive/16rbsqt0DOAAIni4SEra8nKrwZ3O1a872?usp=sharing] to download a file from iCloud to the local computer. This consisted of initially downloading all of the image files for a specific category and uploading it to the Colab notebook at runtime. The code then renamed these files according to the food category they belonged to, since the files all had "IMG" in their paths when uploaded initially. Since the Colab notebook was only able to download 10 files at a time, we proceeded for the 80 images of each category and downloaded them by tens.

After this step, we created another Colab notebook [https://colab.research.google.com/drive/1TOP0yxfCzVgRHt6Vz_1aJy56_UxKZXaj?usp=sharing] that handled the generation of the model and its training on the dataset. The dataset, already split into training, validation, and test, was inputted into the notebook as zip files for extraction into their individual datasets. Then, we visualized the images in the dataset by using the PIL library. Preprocessing was performed using ImageDataGenerator and factors such as "rescale" and "rotation_range" for augmenting the dataset. From each of these datagens, individual data generators for training and validation were constructed by using the ".flow_from_directory" tool, with a target size of (224, 224) (a standard image size), and a batch size of 64.

We then proceeded with developing the model. Our first instinct was creating our own model from scratch using personalized Conv2D layers and Dense layers to classify the information. When we trained this model, we noticed that the validation accuracy was maintained at around 60% as the model was trained for more epochs. We figured that this occurred due to the small layer sizes but noting that model size was a primary constraint for deployment as we initially thought of deploying on the Arduino board. Then, we proceeded to use DepthwiseConv2D with a lower alpha since this offered less multiplications than regular convolution layers. However, retraining this model, the validation increased to a similar benchmark. We then realized that transfer learning could help us for our usecase, specifically using a model that has been already trained on a similar dataset. Therefore, we decided to use MobileNetV2 as our classification transfer model, as it has been already trained on a classification dataset of hundreds of objects. To make this more personalized to our task, we incorporated the transfer model as our base model, additionally implementing layers such as GlobalAveragePooling2D and Dropout with a parameter of 0.2. The reason we decided to use Dropout was to prevent any overspecialization of neurons in a layer that could've caused overfitting to occur (learning more about a given class while paying less attention to the others).



Model: "sequential"

Layer (type)	Output Shape	Param #
mobilenetv2_0.35_224 (Functional)	(None, 7, 7, 1280)	410,208
global_average_pooling2d (GlobalAveragePooling2D)	(None, 1280)	0
dropout (Dropout)	(None, 1280)	0
dense (Dense)	(None, 128)	163,968
dense_1 (Dense)	(None, 6)	774

Total params: 574,950 (2.19 MB)
Trainable params: 164,742 (643.52 KB)
Non-trainable params: 410,208 (1.56 MB)

Figure 2: PlateMate Model Architecture

Model Training and Evaluation Results

We trained the model for 20 epochs and used the "categorical crossentropy" loss function for our training procedure. The optimizer we used was Adam with a default learning rate, as a rate that's higher could limit the model from training optimally and skipping patterns throughout the epochs. The number of epochs parameter required a bit of experimentation to decide the optimal training amount as we wanted to limit the effects of overfitting. Our final training accuracy came out to be 100% with a loss of 0.0116 while our final validation accuracy came out to be 100% with a loss of 0.0054.

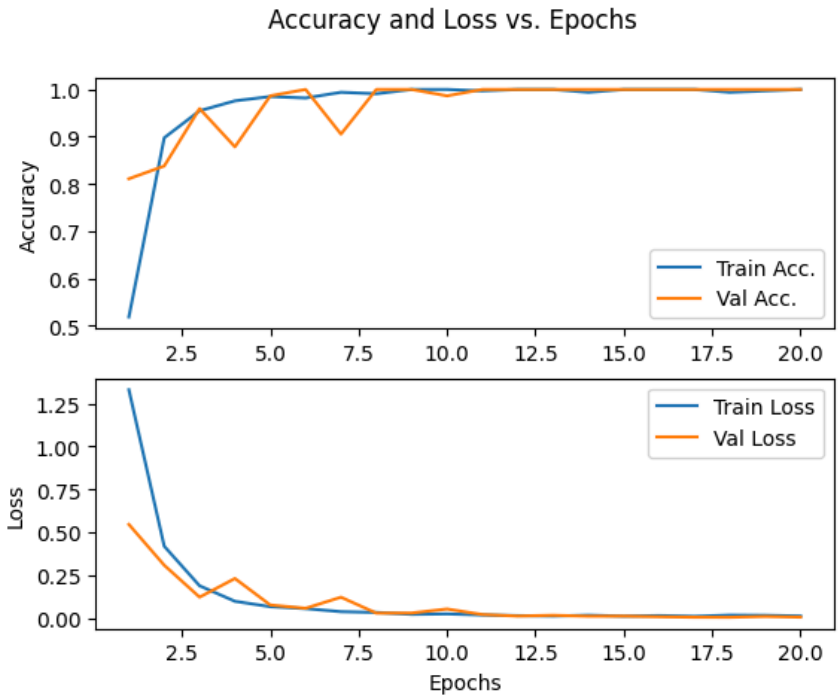


Figure 3: Accuracy and Loss Plots

We then created a test generator using a similar procedure for the training and validation generators, and incorporated this into the "model.evaluate" function. After running this, we noticed our test accuracy came out to 100% and our loss to be 0.00249.

As the model size was particularly large, we then proceeded to quantize this down using TensorflowLite. We first converted the model into its equivalent TFLite model and using integer quantization through making a representative dataset, we got the model size down 0.8 MB. The initial size was 6.3 MB, highlighting an 87% decrease in model size.

Hardware Deployment

Our initial approach was to create a 2-model network where the first model was to implement object detection using OpenCV and then draw bounding boxes on each of the items on the plate. Then, these relative images would pass down to the second model, which would classify the item according to six categories stored in a dictionary with their nutritional content. This we struggled with getting the model size down to fit onto the Xiao or the Arduino, with a max model size of roughly 1 MB. Thus, we proceeded with developing an application that runs on one's phone. Eventually switching to only the classification task, we initially thought about using Xcode or Android Studio to create our mobile app. However, neither of us had prior knowledge developing mobile applications and when we first made a rough framework of the desired app, errors we didn't know how to fix started to show. Then, we discovered Tensorflow.js, a library that could be used on HTML+Javascript web applications and had similar uses when it came to deploying models on hardware. Given that we both had somewhat of an understanding of HTML, we decided to proceed this way and learn more about how the intersection of HTML, CSS, and Javascript actually works. This involved watching Youtube videos and walking through tutorials to develop a firm background before I begin writing code.

VSCode was used as the platform we wrote code on, and LiveServer was used to test out our web application. We both discussed first how we wanted to layout our application and the relevant style choices. Since the web app was to be run on the phone, we talked about developing a vertical layout of input where the user would take a picture of each of the 4 items on the plate and input them individually on the app. For each item, we wanted to display their own nutritional content below the image and then at the very bottom of the screen, the aggregate values would be shown. We first wrote code in HTML to layout the screen and assign relative containers to each one of the inputs. We also added a paragraph below the title to explain the purpose of this app for new users.

CSS was used to style the webpage and make it more appealing to the user. Bright colors were used and a font like Times New Roman created a more formal look. Javascript was used to handle the buttons and the inner classification steps. We initialized a dictionary of the nutritional content for each of the 6 categories and created lists of inputs, outputs, and displays to make processing more efficient. Using the tflite.loadTFLiteModel() function, we uploaded the converted model to a variable named "classifier". The quantized model was not used as it was

not required for this web app (the converted model size was already appropriate). The preprocessing stage from the TF.js library involved functions such as resizeBilinear() for resizing the image and div() for normalizing the images before they are fed into the model. Using the predict() function, we got a tensor that had the confidence values for each class and got the index with the max confidence to represent our class for the object. The confidence value was also used in our display.

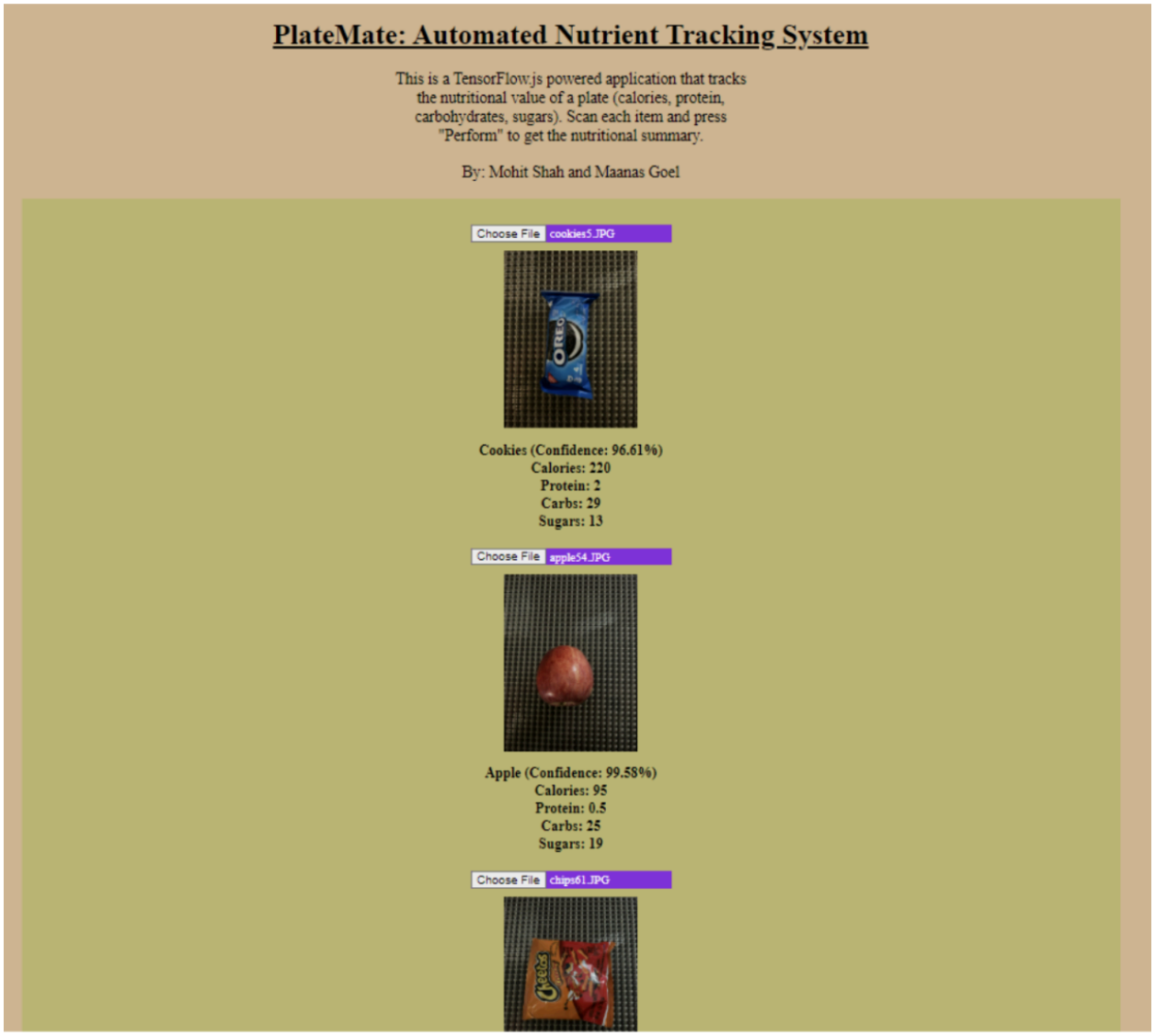


Figure 4: PlateMate Web Application Interface

Github Web Application Link: https://moshah122.github.io/PlateMate/

Github Code Link: https://github.com/moshah122/PlateMate



Figure 4: PlateMate Demo

Full Demo Youtube Link: https://youtu.be/gr-vy6avWxQ

Challenges/ Future Work

One of the main challenges of PlateMate is its limited dataset, which currently focuses on six specific food categories. While the model achieved high accuracy for these specific items, it would struggle to detect more complex foods like salads, sandwiches, or a dish like lasagna. Expanding the dataset to include a wider variety of food types will make PlateMate more effective in real-world scenarios.

Additionally, while PlateMate effectively tracks macronutrients like calories, protein, carbohydrates, and sugars, in the future, PlateMate could also track additional metrics, such as micronutrients like vitamins and minerals. This enhancement would give users a more complete picture of their meal. Another exciting possibility is integrating PlateMate with wearable devices like smartwatches or fitness trackers. By linking the app to these devices, users could monitor their nutritional intake alongside other health metrics, such as physical activity or heart rate, creating a holistic health management system. With this, PlateMate could offer personalized

recommendations tailored to each individual's health goals. These advancements, combined with an expanded dataset and additional nutritional metrics, will make PlateMate even more impactful for promoting healthier eating habits and overall wellness.